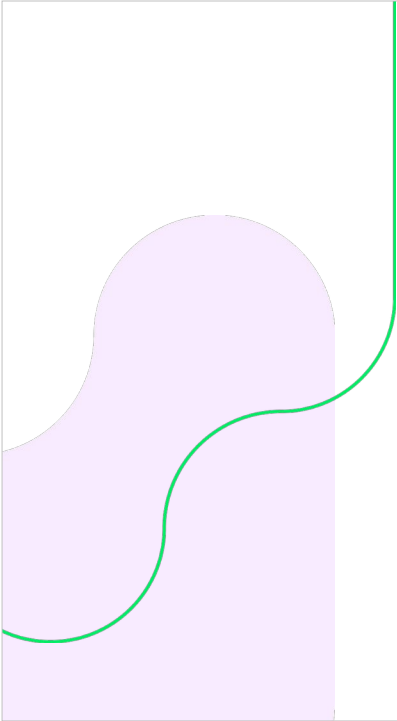




DS110

# Atlas Vector Search

Atlas Search and Vector Search



# Topics we cover

Modern AI Concepts

Vectors and Search

Atlas Vector Search

Use cases

- Semantic search

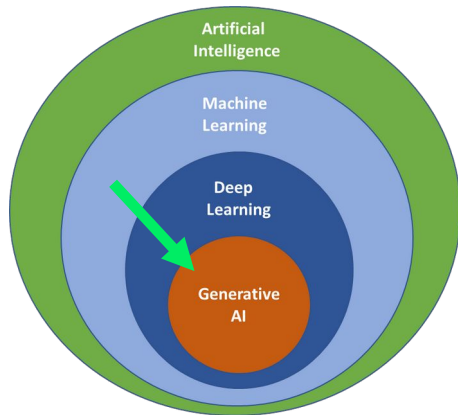
- Retrieval Augmented Generation (RAG)

Atlas Vector Nodes

# Modern AI Concepts



# Generative AI Context



*Generative AI is a type of artificial intelligence system capable of **generating text, images or other media** in response to commands (**prompts**). Generative AI models learn the patterns and structure of their input training data and then generate new data that have similar characteristics.*

*In prompt engineering, the description of the task that the AI is supposed to perform is embedded in the input, e.g., as a question, rather than being explicitly provided.*

Artificial Intelligence -> App that has been programmed to play chess

Machine Learning -> App that has been programmed to automatically improve its chess-playing ability

Deep Learning -> App that has been programmed to automatically improve its chess-playing ability using artificial neural networks

Generative AI -> App that can tell you what it read on the internet in 2021 if you give it the right prompt.

# What is an LLM ?

- **GenAI** uses “Large Language Models” or **LLMs** that provide a general-purpose AI “brain,” no custom building needed for each project.
- LLMs are used for the “generative” functions, for example, creating text, image, write code or video.
- LLMs are the results of **training** on enormous amounts of public data to learn patterns between words, images, videos, or other data.
- GPT-3.5/4 is an LLM which powers the **GenAI** application, ChatGPT.
- Open Source LLMs are available
- You can also generate your own LLM

7

Large language models (LLMs) are a class of deep neural network models that have been trained on vast amounts of text data, which enables them to understand and generate human-like text. GPT stands for Generative Pre-trained Transformer.

Examples of Open Source LLM are: LLaMA 2 (by Meta), BERT (by Google), Falcon (by the Technology Innovation Institute of the United Arab Emirates), or Bloom (From Hugging Face).

# How do LLMs work?

- **LLMs** or “Large Language Models” store vector data
- Text is converted into high-dimensional vectors (**embeddings**) to capture the semantic meaning
- This way is possible to **perform complex operations on text** like efficient similarity searches.

“The girl eats apples”



[0.036017633974552155,-0.012951549142599106,0.025213709101080894,-  
0.00448004761710763,-0.05345296859741211,-  
0.059794988483190536,0.07228174805641174,-0.02564867027103901,-  
0.002973624737933278,0.013297193683683872,0.045300304889678955,-  
0.06130218505859375,0.052884191274642944,-0.009298215620219707,-  
0.03140179440379143,0.028864337131381035,0.12371762096881866,0.006000644993  
036985,0.0374615415930748,-  
0.03103935346007347,0.003820091485977173,0.017213117331266403,-  
0.019120076671242714,-  
0.005894902162253857,0.03814784437417984,0.0178730059415102,0.0820373147726  
059,-0.016479313373565674,-0.13019537925720215,-0.031950194388628006,-  
0.0069317808374762535,-  
0.019303856417536736,0.013285274617373943,0.04293294996023178,-  
0.02165823243558407,0.006749731954187155,0.1297864466905594,-  
0.118458558373416,0.041887187744411,0.043751873737373737

8

LLMs, like GPT-4, are proficient in understanding and generating human-like text. They turn text into high-dimensional vectors (also known as embeddings) that capture the semantic meaning of the text. This transformation makes it possible to perform complex operations on text, like finding similar words, sentences, or documents, which are integral to many applications such as chatbots, recommendation engines, and more.

The nature of these vector representations requires an efficient storage solution that can handle indexing and querying the embeddings, which is where vector databases come in. They store these high-dimensional vectors and allow for efficient similarity searches, making them an ideal choice for LLM-based applications.



## What Can GenAI Be Used For

### **Personalized Recommendations**

Curating and targeting content and next-best actions

### **Financial Modeling**

Predicting market trends or performing risk modelling

### **Scientific Research**

Generating structures for drug discovery or proposing theories

### **Entertainment**

Generating music, art, videos or writing books and movie scripts

### **E-commerce**

Generate product descriptions or scores based on reviews or forecasting inventory demand

### **Sentiment Analysis**

Analysing customer call audio recordings, emails etc.

So it's absolutely not just for Chatbots & Code Generation

GenAI is creative and predictive and can offer insights we may have never thought of before.

In the Financial sector, it can help predict market trends or perform risk modeling.

In the scientific research field, it could generate molecular structures based on past research to prototype drugs or propose scientific theories.

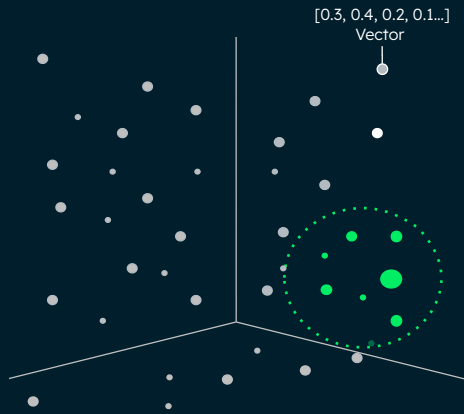
For those in e-commerce, it can generate product descriptions that resonate with potential buyers, give scores based on reviews, and forecast inventory demand.

Also GenAI can analyze audio recordings from customer calls, review thousands of emails, and understand the sentiment behind them. It offers businesses an understanding of their customers' feelings and experiences at a scale once thought impossible.

# Vectors and Search



# Why Vector Search?



Provide **relevant results** from unstructured data

Identify **relations** and **similarities** between inputs and modelled data

11

Unstructured data, like user input, is difficult to match with structured, tagged and organised data.

Vector Search allows for matching data **semantically**.

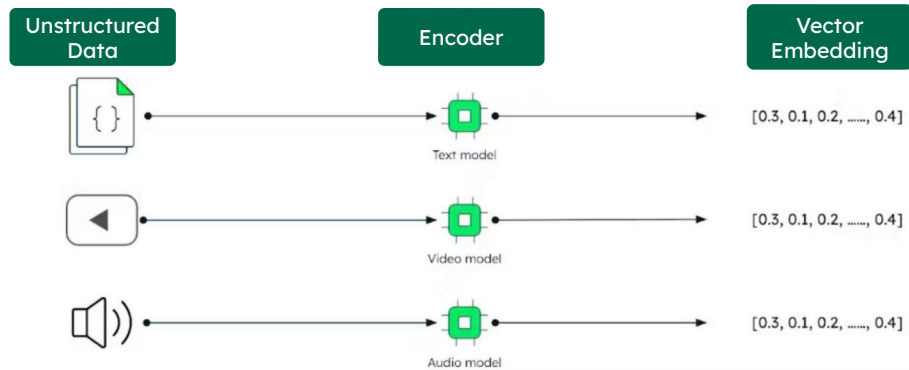
It goes beyond basic keyword matching to find relations between data sets and score them by similarity.

An embedding is a vector (list) of floating point numbers. The distance between two vectors measures their relatedness. Small distances suggest high relatedness, and large distances suggest low relatedness.



# Vector Embeddings

Embeddings are produced by sending data through an encoding model



E.g. OpenAI, Cohere, Anthropic,  
HuggingFace, Vertex, etc.

12

An Embedding is a logical, structured representation of data. What makes it logical and structured is how it is encoded. The Embedding is represented by a Vector of floating point numbers. Each number represents a specific feature of the data. All the numbers together give a representation of the data as a whole.

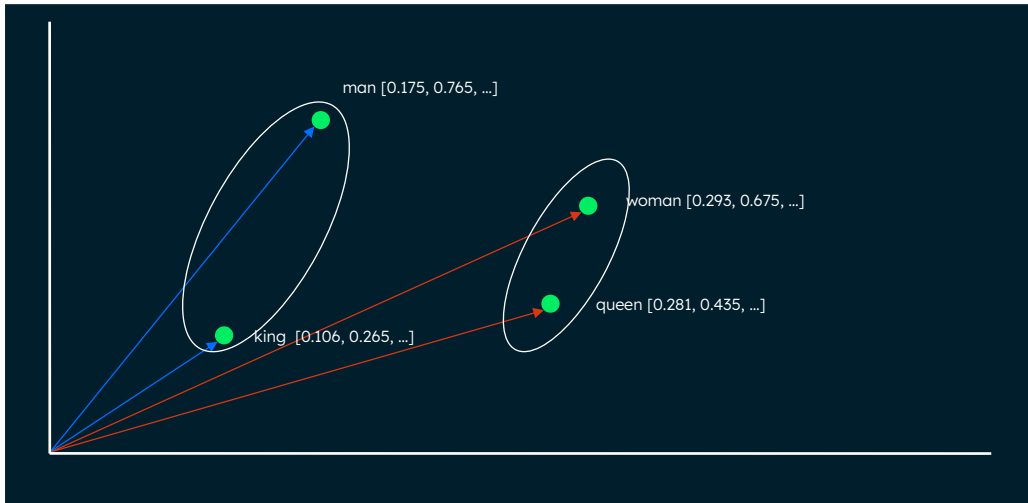
Vector Embeddings are produced by sending data through an encoding model.

Models are trained on large amounts of data. The training process identifies patterns and relationships in the data according to the model's internal parameters.

Once trained, an encoder uses the model to generate a Vector Embedding, which identifies similar patterns and relationships in the unstructured input data.

We store the Vector Embedding along with the unstructured data, for example as an additional field in a MongoDB Document.

# Vector Space



In this example, each word (King, Queen, Man, Woman) has a corresponding *Vector Embedding*. Vector Embeddings can have hundreds to many thousands of individual elements. Each element adds an extra dimension.

Comparing just 2 dimensions here, we see the distance between 'king' and 'man' is small on the *horizontal* axis (X-axis, or dimension/element 0), while the distance between 'man' and 'woman' is small on the *vertical* axis (Y-axis, or dimension/element 1). We might infer that dimension 0 represents gender, while dimension 1 represents social class, but that would be an oversimplification. We normally consider the similarities of vectors *overall*, not on individual elements.

As we compare on more dimensions, our compute cost goes up correspondingly.

Now we can perform a semantic search that is based in *meaning* by calculating the *distance* between vectors in N-Dimensional space.

This example shows words but you can also embed a whole sentence or paragraph into vectors.



# Searching through Vector Space

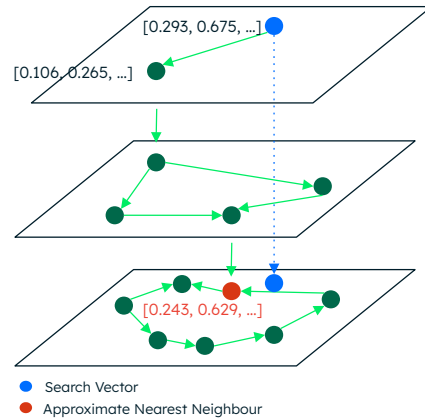
Searching N-Dimensional space is expensive

Exact matches most likely don't exist

'Nearest neighbour' is expensive to find

Solution: find 'Approximate Nearest Neighbour' (ANN) instead

MongoDB Atlas uses HNSW to index Vectors for efficient search



HNSW: Hierarchical Navigable Small World

14

To find a matching Vector, we must search the entire vector space looking for vectors which are similar to ours.

Calculating similarity can mean comparing 'distances' across many hundreds or thousands of dimensions.

Usually, an exact match, or even a 'best' match, would be impractical to find. However, 'approximate' matches are often good enough to be a sensible trade-off.

Vectors must still be indexed so that we can search for them efficiently. Atlas Vector Search uses the "Hierarchical Navigable Small World" algorithm to index and search for Vectors.

NB: Don't create a normal MongoDB index on a field containing Vector Embeddings.



# Searching through Vector Space

When accuracy is needed there is an option to use ENN

ENN (Exact Nearest Neighbor) provides more precision and accuracy on your search.

ENN vector search has advantages when:

- Small-scale vector data
- Recall benchmarking of ANN queries
- Multi-tenant architectures

15

Usually, an exact match, or even a 'best' match, would be impractical to find. So often we use ANN, 'approximate' matches are often good enough to be a sensible trade-off. In other cases, we might want to find those best matches, using ENN capability guarantees retrieval of the absolute closest vectors to your query, eliminating the accuracy limitations inherent in ANN.

ENN vector search can help you unleash a new level of precision for your search and generative AI applications, improving benchmarking and moving to production faster as it always returns the closest vectors to your query, adhering to the specified limit. This can be a powerful tool when building a proof of concept for retrieval-augmented generation (RAG), semantic search, or recommendation systems powered by vector search.

ENN vector search offers advantages in specific scenarios:

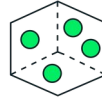
- **Small-scale vector data:** For datasets under 10,000 vectors, the linear time complexity of ENN vector search makes it a viable option, especially considering the added development complexity of tuning ANN parameters.
- **Recall benchmarking of ANN queries:** ANN queries are fast, particularly as the scale of your indexed vectors increases, but it may not be easy to know whether the retrieved documents by vector relevance correspond to the guaranteed closest vectors in your index. Using ENN can help provide that exact result set for comparison with your approximate result set, using jaccard similarity or other rank-aware recall metrics. This will allow you to have much greater confidence that your ANN queries are accurate since you can build quantitative benchmarks as your data evolves.
- **Multi-tenant architectures:** Imagine a scenario with millions of vectors categorized by tenants. You might search for the closest vectors within a specific tenant (identified by a tenant ID). In cases where the overall vector collection is large (in the millions) but the number of vectors per tenant is small (a few thousand), ANN's accuracy suffers when applying highly selective filters. ENN vector search thrives in this multi-tenant scenario, delivering precise results even with small result sets.

# Atlas Vector Search



GA 7.x

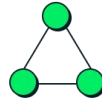
## Atlas Vector Search



**Build** intelligent applications powered by semantic search and generative AI over any type of data



**Store** Vector Embeddings right next to your source data and metadata. Vectors inserted or updated in the database are automatically synchronized to the vector index



**Optimize** resource consumption, improve performance and availability with Search Nodes



**Remove** operational heavy lifting with the battle tested, fully managed Atlas platform

17

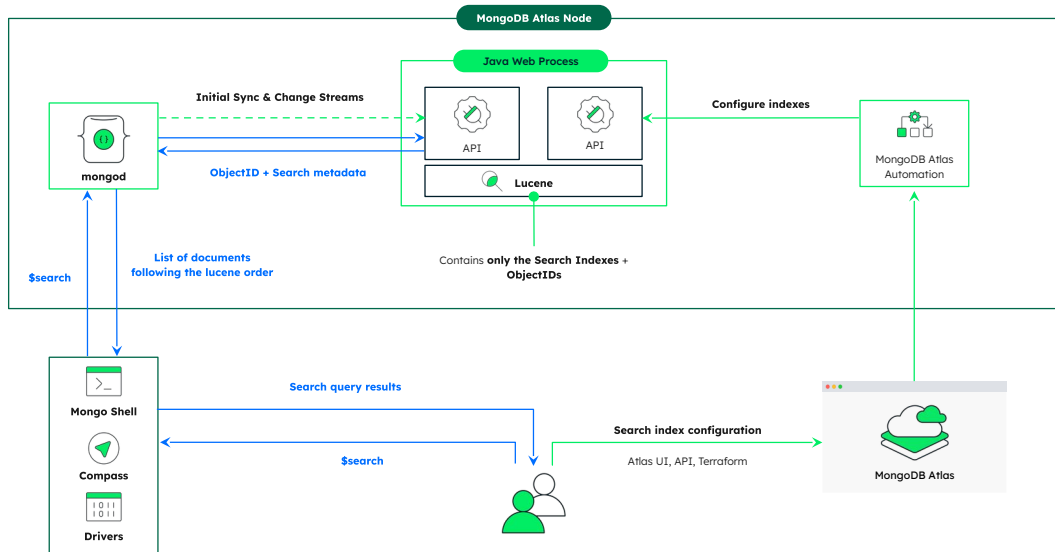
Apache Lucene was evolved for semantic search.

Atlas Vector Search is part of Atlas and Atlas Search. We've designed it on our Atlas Search platform, which uses Lucene, an engine built with semantic search and generative AI in mind. Since Atlas Search is part of MongoDB Atlas, it leverages the power of MongoDB for vector storage right next to the data's source. Vectors inserted and updated in MongoDB are automatically synchronized to the vector index.

We've introduced Search Nodes, dedicated hardware that allows for optimum compute resources and better performance and availability for vector search use cases.

And importantly, Atlas Search with Vector Search removes the operational heavy lifting of keeping the data's source of truth in sync with the search database using the battle-tested, fully managed Atlas Platform.

# Atlas Search Architecture



# A robust ecosystem of AI Integrations



**LlamaIndex**



**OpenAI**



**Hugging Face**



**LangChain**



**cohere**

**NOMIC**



**Microsoft Azure**



**Google Cloud**

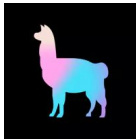
MongoDB offers a robust ecosystem of AI integrations, allowing you to integrate with their favorite LLM or framework.

(we have some great blogs with examples using some of these integrations up on our website if you're interested in learning more)

# Classic Generative AI

LLM Integration Frameworks

 **Semantic Kernel**



**Prompt**



**Generated Response**



LLM Providers



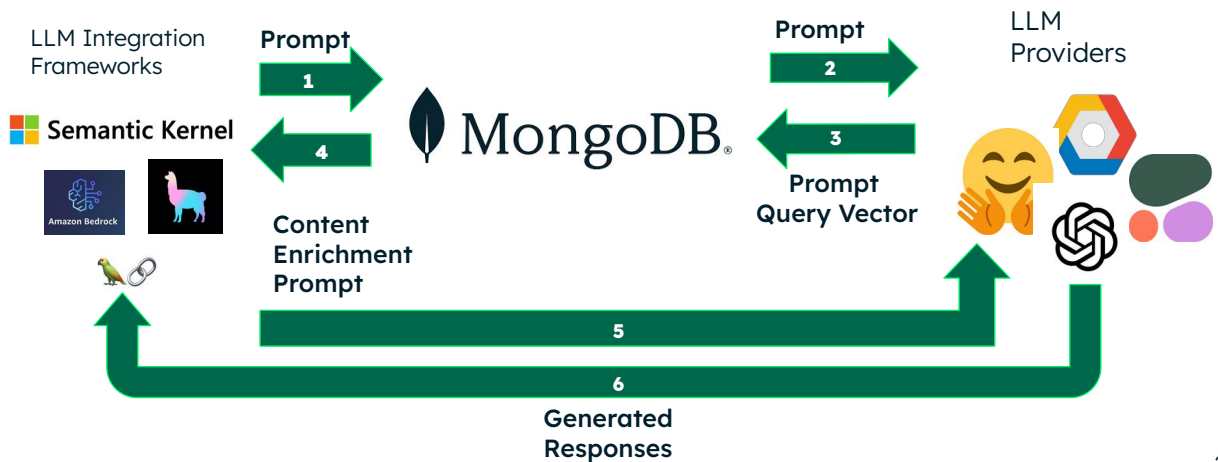
Gen AI ecosystems include applications built on LLM Integration frameworks that interact with the LLM providers based on prompts created by the user input.

The LLM provider uses its internal data and data model to send a generated response based on the received prompt.

But currently these LLMs have some limitations:

- Stale data (from the latest data they used for training the model) - i.e., data only up to 2020, no actual year
- They don't include internal data based on our business. So if we want we need to provide context by providing our data in the Prompt.
- Sometimes the generated response is not accurate. This is known as Hallucinations (false or wrong responses due to lack of data, not having current data, or errors)

# Our place in the AI ecosystem



21

MongoDB can provide now the context and include our business data on top of the LLM provider.

1. User application (LLM Integration Frameworks) send the prompt to MongoDB
2. MongoDB sends this prompt to the LLM provider
3. The LLM provider generates the Query Vector and sends it back to MongoDB
4. MongoDB looks for vectors and generates the additional content enrichment to the prompt, then sends this to the application.
5. The application now can send this final content enrichment prompt to the LLM provider to get generated responses.

# Vector Search - Data Structure



## Sample Document

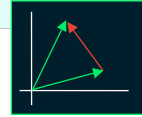
```
_id: ObjectId('62f13a3fe7321ca47aecb216')
symbol: "ABMD"
quarter: 4
year: 2021
date: 2021-04-29T20:10:40.000+00:00
content: "Operator: Ladies and gentlemen, thank you for standing by, and welcome..."
content_embeddings: Array
  0: 0.03898080065846443
  1: -0.05879044905304909
  2: 0.04323238879442215
  3: -0.021337900310754776
  4: -0.036346953362226486
  5: 0.028689613565802574
  6: -0.03514527902007103
  7: -0.07414846867322922
  8: -0.00993054173886776
  9: 0.007234036456793547
  10: -0.03197460621595383
```

Document with an embedding based on the content field.

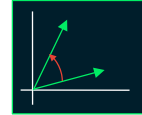
In the Sample dataset, under the sample\_mflix DB, review the embedding\_movies collection to view this example

# Similarity Functions

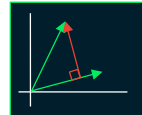
**Euclidean** - *distance* between two vectors



**Cosine** - *angle* between vectors



**Dot Product** - *angle* between vectors multiplied by *distance* between vectors



MongoDB Vector Search supports these 3 similarity functions

- **Euclidean** is good for cases where we want to find similar data, which has been encoded to occupy a specific region in vector space.
- **Cosine** is good for identifying outliers, which would have a large angular distance from other data, regardless of magnitude.
- **Dot product** is good for recommendation systems, where similar items would have a small angular difference, but magnitude matters.

Best results come from using the similarity function recommended by the creator of the Model.

# Vector Quantization

Reduce size of high-dimensional data using compression

- **Scalar Quantization**

- Approximately 1/3.75th of original index size
- Bins values based on max and min

- **Binary Quantization**

- Approximately 1/24th of original index size
- Translates values to 0 or 1.

Vector quantization is a data compression technique that approximates high-dimensional vectors with a smaller, finite set of representative vectors.

This approach reduces the memory footprint of embeddings, which lowers infrastructure costs and enables faster similarity searches.

Quantization improves scalability, though it comes with an expected reduction in accuracy.

- **Scalar quantization:**

- Identifies the maximum and minimum values for each dimension of the indexed vectors to establish a range of values for the given dimension. This range is then divided into equally sized intervals or bins. Each float value is then mapped to a bin, converting the continuous value into a discrete integer.
- In Atlas Vector Search, this quantization reduces the vector embedding's RAM cost to about one fourth ( $1 / 3.75$ ) of the pre-quantized cost.

- **Binary quantization:**

- Assumes a midpoint of 0 for each dimension. Then each value in the vector is compared to the midpoint and assigned a binary value of 1 if it's greater, or 0 if it's less than the midpoint value.
- In Atlas Vector Search, this quantization reduces the RAM cost of the vector embedding to one twenty-fourth ( $1/24$ ) of the pre-quantization cost.
- The reason it's not 1/32 is because the data structure containing the Hierarchical Navigable Small Words makes its own adjustments from the

# Vector Search - Index Definition



## Index Definition

```
{
  "fields": [{
    "path": "content_embedding",
    "type": "vector",
    "numDimensions": 1536,
    "similarity": "<euclidean | dotProduct | cosine>",
    "quantization": "<binary | scalar>",
  },
  {
    "path": "symbol",
    "type": "filter"
  },
  {
    "path": "date",
    "type": "filter"
  }
  ]
}
```

You need to create the vector index in Atlas in order to use the Atlas Vector Search. Vector index has to be defined based on this syntax. The type is vector. The path defines the field containing the embedding in our collection. The number of dimensions are based on the embeddings. The similarity is the function that will be used to find similarities. The quantization is the method that will be used to compress the vectors. Additional filters can also be defined as optional parameters.

# Atlas Vector Search - Query

```
db.collection.aggregate([
```

```
  "$vectorSearch": {
    "exact": false,
    "index": "vector_index_name",
    "queryVector": [ 0.03898080065846443, ... ],
    "path": "content_embedding",
    "limit": 5,
    "numCandidates": 150,
    "filter": {
      $and: [{symbol: "ABMD"},
             {quarter: 4},
             {year: {$lte: 2021}}]
    },
  },
])
```

```
_id: ObjectId('62f13a3fe7321ca47aecb216')
symbol: "ABMD"
quarter: 4
year: 2021

content: "Operator: Ladies and gentlemen, thank

content_embedding : Array
  0.03898080065846443
  -0.05879044905304909
  0.04323238879442215
  -0.021337900310754776
  -0.036346953362226486
  0.028689613565802574
  -0.03514527902007103
  -0.07414846867322922
  -0.00993054173886776
  0.007234036456793547
  -0.03197460621595383
```

When using the Vector search, you define an aggregation pipeline with the \$vectorSearch stage.

This stage includes the vector index to be used, the query vector, and the path as the field in the collection containing the embeddings.

The query vector must be the result of generating an embedding using the same model which generated the embedding in the 'path' field.

Additionally we can apply filters on **boolean**, **numeric** and **string** values (dates or timestamps are not supported). Filtering occurs *before* the vector search is run.

You must define the fields you will filter by when you define the Vector Search index.

**limit** specifies the number of documents to return

**numCandidates** specifies the number of documents to consider.

**exact** is a boolean that indicates if the vector search is using ANN Approximate (false) or ENN Exact (true). If omitted, it is false (ANN) by default,

Atlas Vector Search assigns a score, in a fixed range from 0 to 1 only, to every document that it returns.



## Exercise - Reviewing the collection

Using Atlas, inspect the `sample_mflix.embedded_movies` collection

```
> use sample_mflix  
> db.embedded_movies.findOne()
```

Identify what field contains the embeddings of the movie plot

Use the MDB documentation and find details about the embedding model used to create the field.

Review [https://www.mongodb.com/docs/atlas/sample-data/sample-mflix/#std-label-mflix-embedded\\_movies](https://www.mongodb.com/docs/atlas/sample-data/sample-mflix/#std-label-mflix-embedded_movies)



# Exercise - Creating the index

Create a vector index for the `embedded_movies` collection using euclidean similarity and 1536 dimensions

## Create a Vector Search Index



### Configuration Method

Select how you would like to build and customize your search index. You can also create, edit, and manage search indexes using the Atlas API.

#### NOTE

At this time, search indexes cannot be created for time series collections.

### Atlas Search

#### Visual Editor

Learn about index definitions in a more guided experience.

#### JSON Editor

Edit the raw index definition with an embedded JSON editor.

### Atlas Vector Search

#### JSON Editor

Create a vector search index definition with an embedded JSON editor.

[View Atlas Vector Search Docs](#)

### Database and Collection

Search for database or collection...

- > sample\_airbnb
- > sample\_analytics
- > sample\_geospatial
- > sample\_guides
- ▼ sample\_mflix
  - ☐ comments
  - ☒ embedded\_movies

### Index Name

vector\_index

```
1 {
2   "fields": [
3     {
4       "numDimensions": 1536,
5       "path": "plot_embedding",
6       "similarity": "euclidean",
7       "type": "vector"
8     }
9   ]
10 }
```

Cancel

Next

29

```
{
  "fields": [
    {
      "type": "vector",
      "path": "plot_embedding",
      "numDimensions": 1536,
      "similarity": "euclidean"
    }
  ]
}
```



## Exercise - Using Vector Search

Let's test the new index:

Query the ***embedding\_movies*** collection using the ***aggregate*** command with ***\$vectorSearch*** to find 5 movies similar to this embedding in your Virtual Lab:

```
/root/query_embedding.js
```

Project the plot, title, and score to see the similarity function matches

Change the number of candidates to see if the results change

30

```
load('/root/query_embedding.js');           // creates a local
const called vector_embedding_demo
db.embedded_movies.aggregate([
  {
    "$vectorSearch": {
      "exact": false,
      "index": "vector_index",
      "path": "plot_embedding",
      "queryVector": vector_embedding_demo,
      "numCandidates": 150,
      "limit": 5
    }
  },
  {
    "$project": {
      "_id": 0,
      "plot": 1,
      "title": 1,
      "score": { $meta: "vectorSearchScore" }
    }
  }
])
```

# Vector Search Use Cases





# Two critical categories of use cases

**Semantic Search:** Searching and finding relevant results from unstructured data, based on semantic similarity (meaning and relationship between search terms)

**Retrieval Augmented Generation (RAG):** providing the context of proprietary data using Vector Search to prompt engineer accurate, relevant responses for LLM-based applications (AI use-cases)

32

So circling back to the use cases Vector Search addresses, there are two key ones for businesses: Semantic Search

- So obviously Keyword search is useful when you're searching for something known in a article or table, but what about for all the other use cases?
- What if you're trying to search through a variety of rich media categories to find what you need, including videos, books, social media posts, PDFs, audio files and more? And how do you ensure you receive relevant results?
- Vector Search is a critical capability that addresses this need - Searching and finding relevant results from unstructured data, based on semantic similarity

Retrieval Augmented Generation (or RAG)

- RAG is about providing the context of proprietary data to engineer accurate, relevant responses for LLM-based applications. RAG enables these AI and generative AI use-cases.
- The need we've heard from our customers is augmenting LLM apps with their data, being able to add and index vectors to their existing collections without the need for changing their data model or having to adopt a new tool.



# Semantic Search

Find similarities based on meaning

- Different content formats: images, text, etc.
- Without synonyms, tags, etc.

Example - Retail business searches on product images:

- Green and white coat
- Adidas running shoes
- Black Nike backpack with red logo

# Example - Semantic search



### vector\_search.products

STORAGE SIZE: 323.27MB LOGICAL DATA SIZE: 424.15MB TOTAL DOCUMENTS: 44441 INDEXES TOTAL SIZE: 1.64MB

**Find** Indexes Schema Anti-Patterns Aggregation Search Indexes

Filter Type a query: { field: 'value' }

```
{
  "discountPercentage": 14,
  "averageRating": 3.48
}
```

```
{
  "_id": ObjectId('649422ed2a2e9518dc54b781'),
  "imageFile": "images/56332.jpg",
  "imageVector": Array,
  "price": 11.38,
  "discountPercentage": 14,
  "averageRating": 4.27
}
```

### Demo: MongoDB Atlas Vector Search on Product Images

Product:

**Search Results**

10 results were found for the search for white adidas sport shoes



Image File: images/34830.jpg

- The data in the database:
- Price: \$58.2
- Average Rating: 3.67
- Discount Percentage: 19%
- Search score: 0.6489816707611084
- Embeddings (only 5 dimensions displayed): [0.40726345777511597,0.7631579041481018,-0.09529753029346466,0.8052816987037659]



Image File: images/26326.jpg

- The data in the database:
- Price: \$94
- Average Rating: 4.64
- Discount Percentage: 7%
- Search score: 0.6445645093917847
- Embeddings (only 5 dimensions displayed): [0.3808577050899463,1.0294101028442383,-0.1581447422504425,0.7855895161628723,0.]

34

In this Example we have a products collection, where the image of the product has been encoded into a vector.

This way users can use semantic search finding different articles: shoes, sports shoes, t-shirt, long sleeve t-shirt, bag, etc. and also define color, brands, etc.

This search adapts better to the consumers than the plain text search that maps product description, tags, or product filter fields.

# Example - Semantic search



```
connection = pymongo.MongoClient(mongo_uri)
product_collection = connection[db][collection]
preTrainedModelName = "clip-ViT-L-14"
model = SentenceTransformer(preTrainedModelName)
@app.route('/search', methods=['GET'])

def search():
    vector_text = request.args.get('vector', default=None, type=str)
    vector_query = model.encode(vector_text).tolist()
    pipeline = [
        {
            "$vectorSearch": {
                "index": "default",
                "path":
                    "queryVector": vector_query,
                    "numCandidates": 150,
                    "limit": 10
            },
        },
        {
            "$project": {
                "imageVector": {"$slice": ["$imageVector", 5]},
                "imageFile": 1,
                "price": 1,
                "discountPercentage": 1,
                "averageRating": 1,
                "_id": 0,
                'score': {
                    '$meta': 'searchScore'
                }
            }
        }
    ]

    docs = list(product_collection.aggregate(pipeline))
```

Encoding the user search

Using \$vectorSearch with the image vectors in the product catalog.

35

Review the full demo here: <http://tinyurl.com/VectorSearchImagesDemo>

In this Example, the collection with products was load and the product images (png files) were encoded using a HuggingFace transformer "clip-ViT-L-14".

So in the Atlas Cluster, there is a collection called products with all the images and the image embeddings (in the field **imageVector**)

An Atlas Vector search index on this field is required to be created (in this case the name is default)

In the code we specify the URI to connect to MDB, the collection and DB, the Embedding model ( this is required as the user input will be encoded based on the specified model)

We can run the search function, encoding the user input and passing this user embedded query into the pipeline with \$vectorSearch.

Recommended blog: <https://www.mongodb.com/developer/products/atlas/multi-modal-image-vector-search/>



# Exercise: Movie Recommendations

Use the Sample data set (**sample\_mflix.movies\_embeddings** collection) to build a movies recommendation app using the existing code:

Review the **movie\_recommendations** code

Replace the parameters in the code to include MongoDB URI and API\_Key\*

Test the functionality by searching for a movie plot

36

\*API Key will be provided for the Instructor

Note for instructor: Use the API Key token provided by the Curriculum team

Setup:

1. Update **params.py** with your MongoDB connection string and MDB TOKEN
2. Create a new Python environment: **python3 -m venv env**
3. Activate the new Python environment: **source env/bin/activate**
4. Install the requirements: **pip3 install -r requirements.txt**

Test:

Via bash Terminal: **python3 recommend.py -t "A poisoned man fights to find the antidote"**

Via bash Terminal: (using <http://127.0.0.1:5000>): **python3 app.py**

**NOTE: Attempting to run the command `python3 app.py` within the VSCode integrated terminal will cause the Instruqt instance to crash. Use the bash terminal tab instead.**

Example using Huggingface:

<https://www.mongodb.com/developer/products/atlas/building-generative-ai-applications-vector-search-open-source-models/>



# Retrieval Augmented Generation (RAG)

The retrieval-augmented generation (RAG) architecture was developed to address some of the current LLM limitations

- Hallucinations
- Stale data
- No access to users local data
- Token limit

RAG provides context to the LLM using retrieved documents from a initial vector search.

37

LLMs have revolutionized the field of natural language processing, but they do come with certain limitations:

- **Hallucinations:** LLMs sometimes generate factually inaccurate or ungrounded information, a phenomenon known as “hallucinations.”
- **Stale data:** LLMs are trained on a static dataset that was current only up to a certain point in time. This means they might not have information about events or developments that occurred after their training data was collected.
- **No access to users’ local data:** LLMs don’t have access to a user’s local data or personal databases. They can only generate responses based on the knowledge they were trained on, which can limit their ability to provide personalized or context-specific responses.
- **Token limits:** LLMs have a maximum limit on the number of tokens (pieces of text) they can process in a single interaction. Tokens in LLMs are the basic units of text that the models process and generate. They can represent individual characters, words, subwords, or even larger linguistic units. For example, the token limit for OpenAI’s gpt-3.5-turbo is 4096.

RAG uses vector search to retrieve relevant documents based on the input query. It then provides these retrieved documents as context to the LLM to help generate a more informed and accurate response.

Instead of generating responses purely from patterns learned during training, RAG uses those relevant retrieved documents to help generate a more informed and accurate response.



# Why RAG?

Sending the question as-is directly to the LLM

L

You

As my bank manager application, provide a short summary list of the names and totals of my main categories of spending over the last month.



ChatGPT

I'm sorry for any confusion, but as a text-based AI developed by OpenAI, I don't have access to personal data about individuals unless it has been shared with me in the course of our conversation. I am designed to respect user privacy and confidentiality.

If you provide a list of your spending categories and their corresponding totals, I can help you summarize or analyze that information. However, I won't be able to retrieve any personal data unless it has been shared with me in the course of our conversation. Please ensure not to share any sensitive personal information.

Standard Prompt interaction with LLM

# Why RAG?

The application retrieves the user's recent bank statement items from the database, injects this info (plus some control instructions) into the prompt and sends it to the LLM

Augmenting the question then sending as the new prompt to the LLM

**You**

CONTEXT:

- 2024-01-03 Online Retailer Shop - 58.76
- 2024-01-07 City Electric - 60.00
- 2024-01-12 SuperMarket Groceries - 85.25
- 2024-01-15 Downtown Diner - 20.40
- 2024-01-19 Charlie's Pizzeria - 43.50
- 2024-01-23 Bakery & Coffee shop- 7.35
- 2024-01-27 Local Gym - 30.00
- 2024-01-30 Waterworks utility - 45.00

QUESTION:

Using my list of transactions in the CONTEXT above, as my bank manager application, provide a short summary list of the names and totals of my main categories of spending over the last month.

RESPOND WITH THE SUMMARY LIST ONLY AND DO NOT INCLUDE AN INTRODUCTION TEXT, INDIVIDUAL BREAKDOWNS, NOTES OR EXPLANATIONS.

**ChatGPT**

Online Retailer Shop: \$58.76  
City Electric: \$60.00  
SuperMarket Groceries: \$85.25  
Downtown Diner: \$20.40  
Charlie's Pizzeria: \$43.50  
Bakery & Coffee Shop: \$7.35  
Local Gym: \$30.00  
Waterworks Utility: \$45.00

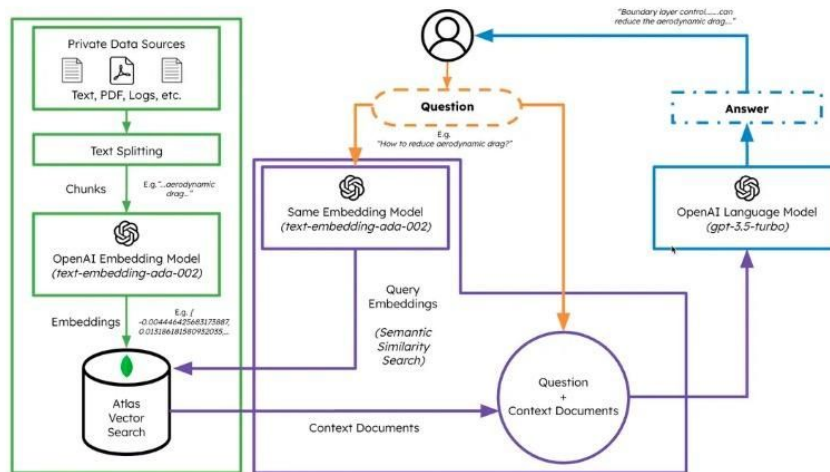
You can see inside the blue boxes, the additions the application makes to the prompt's text that it sends.

The application retrieves the user's recent bank transactions from the database and then augments the prompt with the historical context. This is a common pattern. An application is basically asking an LLM to make sense of the data it provides.

The application also does something else, which is common when engineering the prompts it sends. It injects some control instructions into the prompt to coerce the LLM to build its response in a specific format. For example, the app says to the LLM not to include any explanations in its response.

LLM now returns an accurate response relevant to the user's bank transaction history for the user which the application is asking on behalf of.

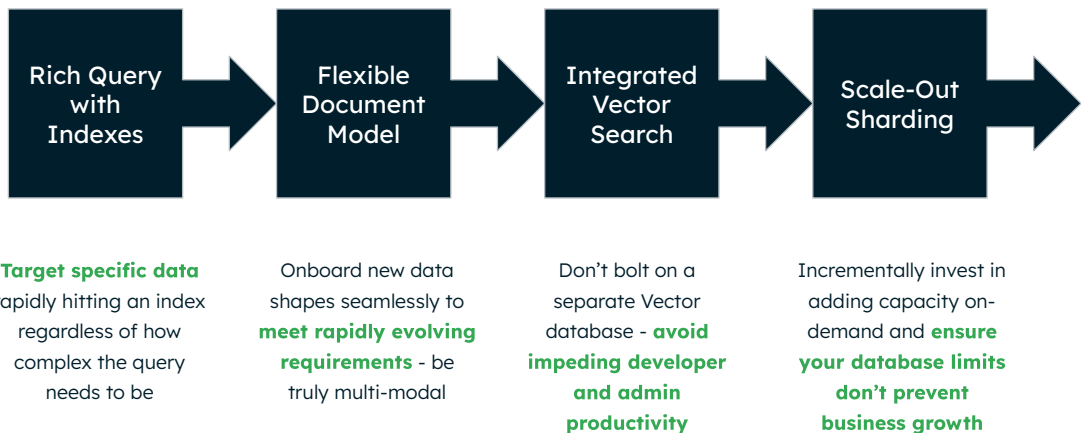
## Example - RAG app architecture



Review this example using LangChain: <https://www.mongodb.com/developer/products/atlas/rag-atlas-vector-search-langchain-openai/>

LangChain is a framework designed to simplify the creation of LLM applications. It provides a standard interface for chains, lots of integrations with other tools, and end-to-end chains for common applications. This allows AI developers to build LLM applications that leverage external sources of data (for example, private data sources).

# Database requirements for effective RAG



For Retrieval Augmented Generation, you need a database where you can query the relevant data in milliseconds by always hitting indexes, regardless of the query complexity required.

You need the agility to iterate quickly, adding new requirements seamlessly by having a flexible document model.

You want powerful Vector Search integrated directly into the database for maximum developer productivity.

You must be able to continuously scale out your database, via Sharding, to ensure you can maintain business growth.

MongoDB Atlas meets the four essential required capabilities.

# Atlas Developer Data Platform

Document Model & Unified API



OLTP



TIME  
SERIES



FULL-TEXT  
SEARCH



ANALYTICS



STREAM  
PROCESSING



VECTOR  
SEARCH

Multi-Cloud Scale, Resilience, Performance, & Security



Atlas can provide a unified approach to building applications.

This developer data platform allows us to use more than just Atlas Search and Vector Search.

We can use Atlas Database triggers to automate triggers with change events to encode any new movies added to the collection.

Atlas platform offers much more than just Atlas Search and Vector Search.

# Exercise - Part 1:

## Setting Triggers to encode new documents



Create the Trigger:

Add a Trigger called *AddEmbeddingsTrigger* and configure the trigger settings:

Select your Cluster Name.

Database Name: sample\_mflix

Collection Name: embedded\_movies

Operation Type: Insert Document

Full Document: On

Replace the code with the provided Trigger code (next slide).



# Trigger code

Your instructor will provide this code to you.

Replace `<API_KEY>` with  
your OpenAI `API_KEY`

```
exports = async function(changedEvent) {
  const _id = changedEvent.fullDocument._id;
  const plot = changedEvent.fullDocument.plot;

  // Get the MongoDB service you want to use (see "Linked Data Sources" tab)
  // Note: In Atlas Triggers, the service name is defaulted to the cluster name.
  const serviceName = "cluster0";
  const database = "sample_mflix";
  var collName = "embedddo_movies";

  // Get a collection from the context
  var collection = context.services.get(serviceName).db(database).collection(collName);

  var MDM_API_KEY = "<API_KEY>"
  const arg = {
    url: "https://ollvectorsearch.openai.azure.com/openai/deployments/text-embedding-ada-002/embeddings?api-version=2023-05-15",
    "Content-Type": ["application/json"],
    "api-key": [ MDM_API_KEY ]
  },
  body: JSON.stringify({
    input: plot,
  })
  };

  const response = await context.http.post(arg);
  // The response body is a JSON binary object. Parse it to JSON.
  const responseBody = EJSON.parse(response.body.text());
  const embeddings = responseBody.data[0].embedding;

  try {
    // Update the document with the new embeddings
    const result = await collection.updateOne(
      { _id: _id },
      { $set: { plot_embedding: embeddings } }
    );
    // Access the number of documents modified
    const modifiedCount = result.modifiedCount;
    //console.log("Modified " + modifiedCount + " document(s)");
    return { result: modifiedCount };
  } catch (err) {
    console.log("Error occurred while executing updateOne:", err.message);
    return { error: err.message };
  }
};
```



## Exercise - Part 2:

# Setting Triggers to encode new documents

Now you can test this trigger by adding a new movie to the collection. Testing the results:

In your terminal, connect to the Atlas cluster

Type the following commands to insert one movie in the collection:

```
use sample_mflix
db.embedded_movies.insertOne({
  "title": "Barbie",
  "plot": "Barbie and Ken are having the time of their lives in the colorful and seemingly perfect world of Barbie Land. However, when they get a chance to go to the real world, they soon discover the joys and perils of living among humans.",
  "imdb": {
    "rating": 6.9
  }})
```

Find the movie by searching the title “Barbie” and check the embedding was included.

```
db.embedded_movies.find({"title": "Barbie"})
```

Depending on the load on the OpenAI server, it may take a few seconds for the update to reflect.

# Atlas Search Vector Nodes



M10 or higher

## Search Nodes

### Resource Optimization and Flexibility

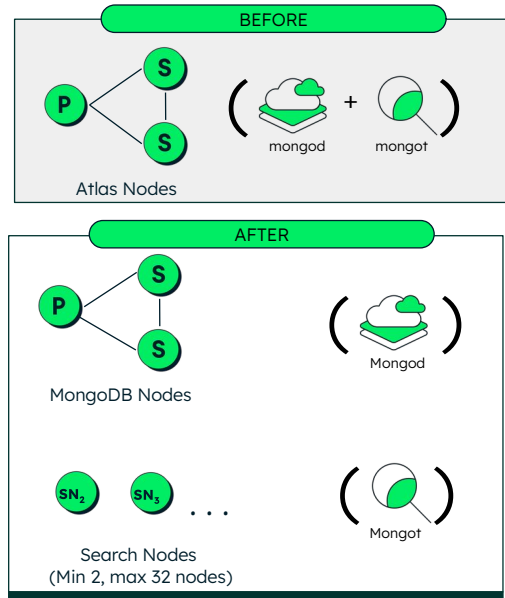
Confidently handle and scale demanding Search workloads

### Better performance

40% - 60% decrease in query time for many complex queries

### Higher Availability

Workload isolation avoids resource contention between database and search



48

From v6.0, Atlas supports deploying search nodes for Atlas Search separately on AWS, Google Cloud and Azure

<https://www.mongodb.com/docs/atlas/cluster-config/multi-cloud-distribution/#std-label-configure-search-nodes>

Create your dedicated cluster



Turn the Workload isolation on



Google Cloud



**Multi-Cloud, Multi-Region & Workload Isolation** (M10+ clusters)  
Distribute data across clouds or regions for improved availability and local read performance, or introduce read-only, analytics and search nodes.  
[Learn more](#)



Turn Search nodes option on



**Selectable nodes for high availability**  
Configure 3, 5, or 7 nodes across multiple regions to better withstand data center outages

★ Recommended region

| Provider | Region                  | Priority | Nodes | Action |
|----------|-------------------------|----------|-------|--------|
| AWS      | N. Virginia (us-east-1) | HIGHEST  | 3     |        |

+ Add a provider/region

Total: 3 Selectable Nodes

**Read-only nodes for optimal local reads**  
Add replicas in additional regions to optimize for local reads in any of your service areas

| Provider | Region | Nodes | Action |
|----------|--------|-------|--------|
|----------|--------|-------|--------|

+ Add a provider/region

Total: 0

**Analytics nodes for workload isolation**  
Isolate analytics queries on nodes that will not contend with your operational workload

| Provider | Region | Nodes | Action |
|----------|--------|-------|--------|
|----------|--------|-------|--------|

+ Add a provider/region

Total: 0

**Search Nodes for workload isolation** NEW

Provide nodes dedicated to search indexes to support your mission-critical workloads. [Learn more](#)

Continue to Cluster Tier to select an appropriate tier for your search workload.

Search Nodes per region:

Region: AWS / N. Virginia (us-east-1)

**Considerations for creating a cluster with Search Nodes**

- You can have a minimum of 2 Search Nodes and up to 32.

☐ I understand and agree to the considerations for creating a cluster with Search Nodes

You can increase and reduce the number of search nodes that you deploy for your cluster when you create or modify a cluster.

You can deploy a minimum of 2 and a maximum of 32 search nodes on you Atlas cluster (charged hourly)

Atlas provides different search tiers. Each search tier has a default RAM capacity, storage capacity, and CPU.

# Quiz Time!





# #1. What specific advantage does MongoDB Vector Search have over pure Vector Databases?

A

Can be hosted in the cloud

B

Can use dedicated hardware

C

Can be accessed via a Unified API

D

Can store vectors alongside other data

E

Can compress vectors

Answer in the next slide.



# #1. What specific advantage does MongoDB Vector Search have over pure Vector Databases?

A

Can be hosted in the cloud

B

Can use dedicated hardware

C

Can be accessed via a Unified API

D

Can store vectors alongside other data

E

Can compress vectors

MongoDB Vector Search's specific advantage is that vectors can be stored alongside other data, even in the same document.



## #2. What are two properties of Vector Embeddings?

A

They are documents in JSON that include metadata

B

They are arrays of semantic strings with synonyms and tags

C

They are arrays of decimal numbers between -1 and 1

D

They are based on a specific embedding model

E

Their dimensionality is fixed to 1536 array items

Answer in the next slide.



## #2. What are two properties of Vector Embeddings?

A

They are documents in JSON that include metadata

B

They are arrays of semantic strings with synonyms and tags

C

They are arrays of decimal numbers between -1 and 1

D

They are based on a specific embedding model

E

Their dimensionality is fixed to 1536 array items

Data is represented in a Vector Embedding as an array of decimals. This is the output of passing the data (text, image, etc) through an embedding model. The number of dimensions, or elements, in a Vector Embedding, depends on the embedding model used to generate it.



### #3. What similarity functions can we use in a MongoDB Vector Search index?

A

Euclidean

B

Hamming

C

Manhattan

D

Cosine

E

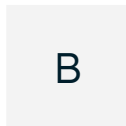
Dot Product

Answer in the next slide.

### #3. What similarity functions can we use in a MongoDB Vector Search index?



Euclidean



Hamming



Manhattan



Cosine



Dot Product

The available similarity functions are euclidean, cosine and dot product.

**SKILL BADGE ASSESSMENT – 10 MINUTES**

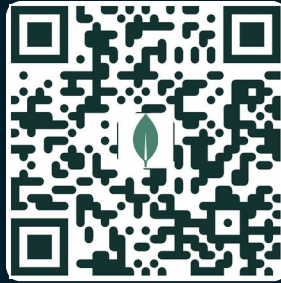


# GenAI: Vector Search Fundamentals

## Questions

MongoDB Skill

Vector Search  
Fundamentals



Take the Assessment



Ask your neighbor



Raise your hand



Let's do it together!

<https://mdb.link/Skill-VectorSearchFundamentals-PS>

# Recap

Atlas Vector Search is an Atlas feature that creates indexes to support Semantic search using Gen AI LLM with Apache Lucene.

Search results are scored to rate how well they match the search.

Vector (embeddings) are stored in the collection for easier storage and retrieval.

Main use cases are semantic search and RAG.